

TECHNICAL ARCHITECTURE · METHODOLOGY · IMPLEMENTATION REPORT

VetGenome Hub

A Complete Genome Analysis Pipeline

ICAR — National Institute of Veterinary Epidemiology and Disease Informatics (NIVEDI) ·
VetGenome Hub, Genomic Surveillance Facility



Five Cooperating Layers

The platform is organised as five cooperating layers. A request flows top-to-bottom while results and status flow back up, and each layer is independently replaceable.

2.1 Architectural layers

Layer	Responsibility	Implemented with
Presentation	Operator dashboard: upload, run, live progress, results, charts, logs, cohort views	Static HTML + CSS + vanilla JavaScript
API	REST endpoints for pipelines, jobs, files, reports.	FastAPI (ASGI) served by Uvicorn
Orchestration	Per-run chain driver, job store, concurrency-bounded executor, restart recovery	Python: custom <code>_run_chain</code> + <code>ThreadPoolExecutor</code>
Tool adapters	Uniform wrappers that build & run each bioinformatics tool	Python adapter contract (tools/<id>/) + Bash
Execution & data	Real tool execution in WSL/conda envs; isolated working dirs; reference databases	Conda/mamba envs · WSL · filesystem · SQLite/MySQL

2.3 Design principles

Principle	What it guarantees
Per-run isolation	Every run computes in <code>work/<run_id>/</code> — two runs can never overwrite each other's intermediates or reports.
Checkpoint / resume	A step whose non-empty output exists is skipped, so an interrupted run resumes instead of recomputing the expensive assembly.
Report-only safety	Flag, don't filter — a non-destructive gate that flags issues and routes for review, never silently discards data.
Honest status	A step is marked done only if it produced credible, non-empty output — 'exit 0 but produced nothing' is reported as a failure.

Chosen for Transparency & Swappability

The stack was chosen for transparency and low operational overhead on a single workstation/server, while keeping each component swappable. Python anchors the bioinformatics ecosystem, FastAPI gives an async REST surface with automatic OpenAPI docs, and conda/mamba pins reproducible toolchains.

3 Languages, frameworks & their rationale

Concern	Choice	Why
Backend language	Python 3.10	Dominant language of bioinformatics tooling; rich ecosystem; easy adapter pattern
Web framework	FastAPI + Uvicorn (ASGI)	Modern, async-capable, automatic OpenAPI schema, minimal boilerplate
Frontend	HTML5 + CSS3 + vanilla JavaScript	No build step, no framework lock-in; loads instantly; trivially deployable as static files
Tool scripting	Bash	Native glue for CLI bioinformatics tools and the vendored downstream modules
Query language	SQL (SQLite dialect; MySQL mirror)	Job & run metadata; relational downstream result mirror
Environment mgmt	Conda / Mamba	Reproducible, pinned bioinformatics toolchains in isolated environments
Host platform	WSL2 (Linux on Windows)	Runs Linux-only bioinformatics tools on a Windows facility workstation

Design note

The frontend follows the VetGenome Hub v1 design system — a warm-white surface with a deep-teal accent (#0F766E) — for visual continuity. It is intentionally a single static interface, with no React/Vue/build pipeline, in line with the project decision to keep the presentation layer dependency-free and instantly deployable on modest facility hardware.

Metadata Stores, Artifacts & Reference Databases

The platform separates metadata — small, relational and queried constantly — from scientific artifacts, which are large files kept on disk. SQLite is the primary metadata store; an optional MySQL/MariaDB instance mirrors downstream module results for cross-sample queries.

4 Stores

Store	Engine	Contents	Role
jobs.db	SQLite	Every tool job: id, tool, mode, params, status, return code, outputs, timestamps	Primary job store
pipelines.db (runs)	SQLite	Run manifests: ordered step list, per-step status, sample, reference, cohort membership	Primary run store
resistome	MariaDB / MySQL (XAMPP)	Schema duplication across sources — rows mirrored from each source report into a common, normalized table structure.	Optional relational mirror
Per-run working dir	Filesystem (work/<run_id>/)	All intermediates & outputs: qc, trimmed, kraken, assembly, aligned, variants, consensus, amr, mge	Scientific artifacts
Consensus library	Filesystem (shared)	Published per-sample consensus FASTAs used by cohort analyses	Cohort input

4.1 Reference databases

Curated reference databases power classification and downstream analysis; all are version-pinned and fetched during environment setup.

Reference database	Used for
Kraken2 Standard-16	Taxonomic classification and contamination profiling
Host index — bovine Bowtie2 index	Host-read removal via KneadData
Clair3 models (r1041_e82_400bps_sup, v500 PyTorch)	Neural small-variant calling for ONT
NCBI AMRFinderPlus,abricate,ResFinder database	Curated acquired AMR genes + point mutations
CARD/RGI · ResFinder · PlasmidFinder · MOB-suite · ISEScan · IntegronFinder · PhiSpy · ABRicate	Used by the MGE-Sift downstream pipeline

Storing artifacts on disk (not as BLOBs) keeps databases small and lets standard genomics tools read files directly; the MySQL mirror is strictly additive — if unreachable, AMR reporting still works from the on-disk JSON reports.

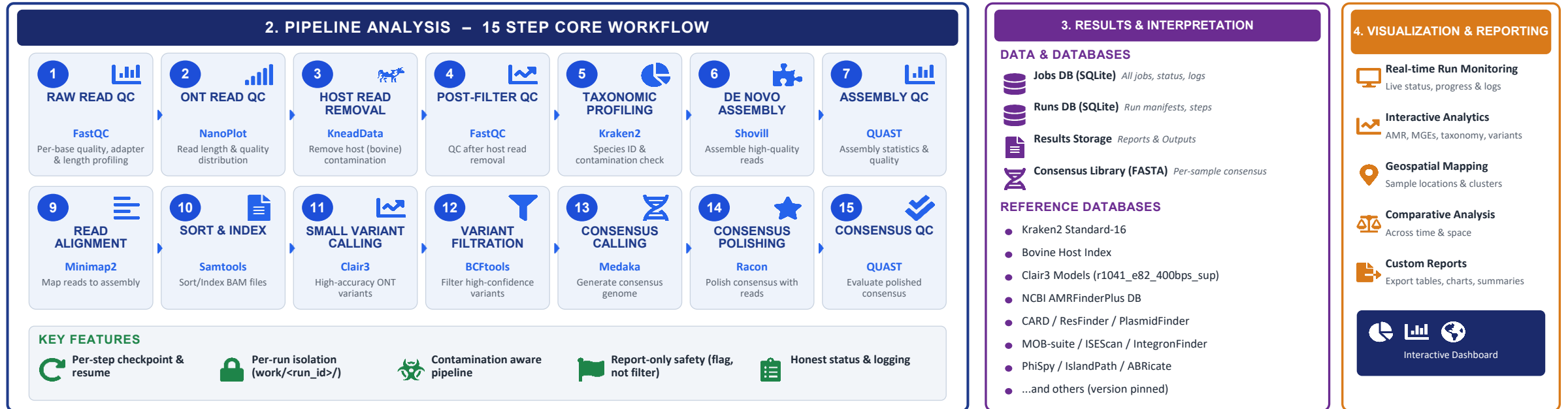
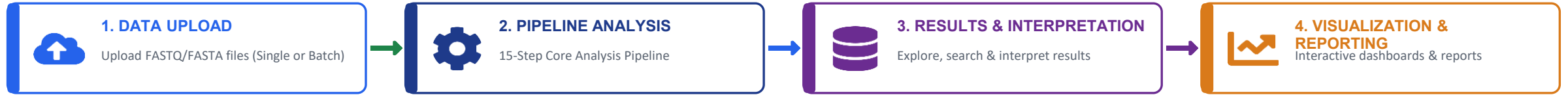


VETGENOME HUB – COMPLETE WORKFLOW

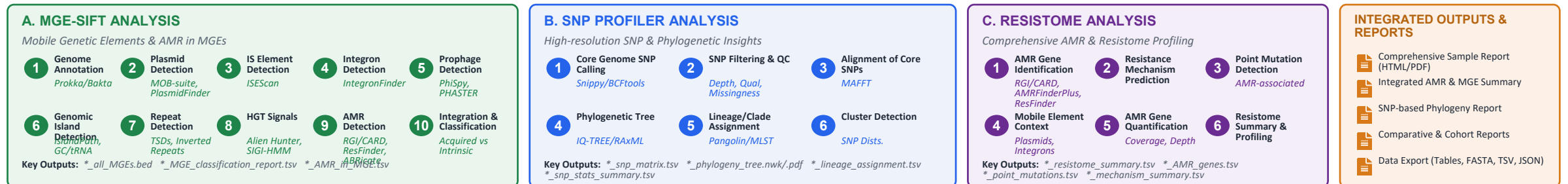
VetGenome Hub

ICAR-NIVEDI

From Data Upload to Genomic Insights



FURTHER DOWNSTREAM ANALYSES (POST CORE PIPELINE)



PLATFORM & INFRASTRUCTURE



Nextflow Workflow Engine



Docker/Conda Containerization



FastAPI (Python) Backend API



SQLite Databases



Web UI (HTML/JS) Dashboard



File Storage (Local/NAS/Cloud)



Secure Access & User Mgmt

LEGEND

→ Data Flow
→ Results Flow

→ Control Flow
→ Metadata Flow

The Ordered Single-Sample Chain (18 Steps)

Each single-sample run executes this ordered chain. Steps are topologically ordered, so every step's input is guaranteed to exist before it runs; the chain is contamination-aware and gated on quality, carrying raw ONT reads all the way to an annotated consensus genome.

#	Stage	Tool	Purpose
1	Raw read QC	FastQC	Per-base quality, adapter & length profiling
2	ONT read QC	NanoPlot	Nanopore read-length / quality distributions
3	Adapter trimming	Porechop_ABI	Detect & remove ONT adapters / barcodes
4	Quality/length filter	Chopper	Drop low-quality and too-short reads
5	Taxonomic classification	Kraken2	Assign reads to taxa; basis for contamination
6	Classification tidy-up	KrakenTools	Extract / format Kraken2 results
7	Classification viz	Krona	Interactive taxonomy composition chart
8	Host removal	KneadData	Remove host (bovine) reads — auto-skipped if host-free
9	De novo assembly	Flye	Assemble cleaned reads into contigs

#	Stage	Tool	Purpose
10	Consensus polishing	Medaka	Neural polishing of the assembly (ONT)
11	Assembly QC	QUAST	Contiguity / completeness (N50, length, #contigs)
12	Read mapping	minimap2	Map reads to reference for variant calling
13	Alignment processing	samtools	Sort / index the alignment
14	Small-variant calling	Clair3	Neural SNV/indel calling (haploid, ONT v500)
15	Structural-variant calling	Sniffles	Detect structural variants
16	Coverage / masking	bedtools	Low-coverage regions for consensus masking
17	Filter & consensus	bcftools	Quality-filter variants; build consensus FASTA
18	Aggregate report	MultiQC	Combine all QC into one dashboard (non-blocking)

Key rationale: contamination-aware routing skips host removal when a sample is host-free; Medaka neural polishing lifts ONT consensus accuracy, improving downstream calls; and bcftools applies quality-filtered Clair3 variants and masks low-coverage regions for a trustworthy consensus.

Operational Today & On the Roadmap

Once a consensus genome exists, operators launch downstream analyses from the dashboard. Below: what is operational today for bacterial isolates, and the planned next step — a viral-genome downstream pipeline on the same orchestrator.

Operational today and in progress (bacterial)

Analysis	Tools / engine	What it produces
Resistome Navigator (AMR)	NCBI AMRFinderPlus (organism-aware)	Acquired genes, chromosomal point mutations & a by-drug-class aggregate; confidence tiers (Core → HIGH, Plus → MEDIUM); JSON (+ optional MySQL); Single / Cohort scopes.
MGE-Sift (Mobile Genetic Elements)-in progress	MOB-suite, PlasmidFinder, ISEScan, IntegronFinder, PhiSpy, RGI/ResFinder/ABRicate, Prokka (own conda env)	Plasmids, IS, integrons, prophages & islands; reports which AMR genes ride on which element; each element acquired (HGT) vs intrinsic, with confidence.
SNPprofiler	bcftools (subset, normalize, filter) + SnpEff/VEP (functional annotation), running on the existing Clair3/bcftools VCF — no new variant calling.	A structured per-isolate SNP report (annotated, flagged, AMR-cross-referenced) and, in cohort scope, a sample × position SNP matrix, pairwise distance matrix, and association results — all JSON, ready for the dashboard's tables and heatmaps.

16 On the roadmap — downstream



Viral genome pipeline — downstream PLANNED

Extend the same orchestrator to viral pathogens: ONT viral read QC, host depletion, reference-guided & de novo consensus, lineage / clade typing and antiviral-resistance markers — surfaced through the existing dashboard, cohort and surveillance views.

Also planned

Durable workflow engine / queue

PostgreSQL metadata + results warehouse

Object storage (S3 / MinIO) with tiering

Surveillance & outbreak dashboards

Live pipeline-graph (DAG) UI